PrecisionAuto Care

Comprehensive Project Review & Viva Revision Guide (SOA / Microservices) FACULTY VIVA PREP

Architecture: Spring Cloud Microservices

Tech Stack: Java 21 LTS • Spring Boot 3.3.4 • Eureka • Gateway

Ports: 8761 (Registry), 8080 (Gateway), 8081-8084 (Services)

1. Project Summary in 4–5 Lines (The Elevator Pitch)

> *"Sir, **PrecisionAuto Care** is an enterprise automotive fleet maintenance platform built on a **Java 21 and Spring Cloud** microservices architecture.*
> *It automates the workshop repair lifecycle and features an intelligent **Collision Elimination Engine** that prevents service bay double-bookings with automated conflict detection.*
> *The backend utilizes **Netflix Eureka** for service discovery, an **API Gateway** for centralized routing, and **OpenFeign** for inter-service communication across 6 independent services.*
> *When a technician completes a repair, the system automatically logs diagnostic history and generates a real-time, tax-calculated invoice (Labor + Parts + 10% Tax).*
> *The platform enforces zero-trust **JWT authentication** with Role-Based Access Control across Customer, Technician, and Admin roles."*

2. Theoretical Purpose: What Real-World Problem Does It Solve?

In commercial fleet operations (logistics, delivery companies, corporate transport), vehicle downtime equals financial loss. This platform eliminates 3 key operational bottlenecks:

- **Physical Bay Constraint Elimination:** Workshops have limited hydraulic lift bays (`BAY-1`, `BAY-2`, `BAY-3`). Manual scheduling results in double-booking collisions. The platform mathematically prevents overlapping reservations using an automated collision avoidance algorithm returning `HTTP 409 Conflict`.
- **Compliance & Diagnostic Audit Trail:** Fleet regulations require audit records for all repairs. Every completed service immutably logs technician ID, date, diagnostics, and replaced parts into a centralized vehicle ledger.
- **Automated Dynamic Financial Settlement:** Eliminates manual accounting delays. The instant a repair is marked complete, labor and parts costs are aggregated with 10% tax into an itemized, payable invoice.

3. Review 1 vs. Review 2: What Changed & Key Deliverables

Use this summary table to demonstrate concrete technical advancement from the preliminary Review 1 milestone to Review 2:

System Capability	Review 1 (Baseline / Initial State)	Review 2 (Advanced Implementation)
Inter-Service Communication	Isolated microservice silos; independent databases; mock static returns; no RPC.	**Spring Cloud OpenFeign RPC Orchestration:** Completing a job in Booking Service triggers automated calls to Record Service & Billing Service simultaneously.
Security & Gateway Layer	Direct unauthenticated endpoint calls; no token verification or filtering.	**Stateless JWT (HMAC-SHA256) + BCrypt:** Custom `JwtAuthenticationFilter` at API Gateway (8080) validates tokens and mutates `X-User-Id` / `X-User-Role` headers.
Bay Conflict Prevention	Basic appointment entries with risk of double-booking service bays.	**Collision Elimination Engine:** Dynamic bay occupancy checks returning standard **HTTP 409 Conflict**; interactive simulator in UI.
Invoicing & Tax Calculation	Static pre-seeded records; manual calculations; no lifecycle.	**Dynamic Financial Engine:** Computes `(Labor + Parts) + 10% Tax` in real time; full payment lifecycle (`PENDING` → `PAID`) with one-click settlement.
Frontend UI & RBAC	Rudimentary mockup without role security; generic styling.	**Enterprise Glassmorphic Dashboard:** 4-tier Google Fonts, vector SVGs, canvas anti-bot CAPTCHA, brute-force guard, dynamic views for Customer, Tech, and Admin.
1-Click Review Startup	Manual command prompt startup for 6 individual services.	**Automated Master Startup (`START_PROJECT_FOR_REVIEW.bat`):** Sequenced orchestration of Eureka, Gateway, 4 services, and Node frontend.

4. "Where is the Backend? Show Me the Working of the Backend!"

When the faculty asks to see the backend, execute these **4 concrete demonstration steps**:

Demonstration Step	Action & URL to Open	What to Say to the Faculty
1. Service Registry (The #1 Proof)	Open `http://localhost:8761` in browser	*"Here is our Netflix Eureka Service Registry. You can see all 5 microservices registered and active with their ports (8080, 8081, 8082, 8083, 8084). None of the URLs are hardcoded; they discover each other dynamically."*
2. Raw Backend REST Endpoints	Open in browser tabs: • `http://localhost:8084/bills` • `http://localhost:8082/bookings` • `http://localhost:8083/records` • `http://localhost:8080/actuator/health`	*"Here are the live REST APIs returning raw JSON directly from each service's embedded Tomcat engine. The frontend simply consumes these APIs through our Spring Cloud Gateway."*
3. IDE Consoles (STS)	Switch to Spring Tool Suite (STS) Boot Dashboard	*"In our IDE, each microservice runs as an independent Java process with its own Spring context, JPA repositories, and configuration."*
4. Live Inter-Service Orchestration	Click **"Complete Job"** on the web UI while watching the `booking-service` console in STS	*"Look at the console log: Booking Service didn't just update its table; it executed synchronous RPC calls via **OpenFeign** to Record Service (8083) and Billing Service (8084) to generate the invoice in real time."*

5. Which Services Are Used in Which Instance (Scenario)?

Scenario / Instance	Services Triggered & Ports	Technical Workflow Description
1. User Login	`API Gateway (8080)` → `Auth Service (8081)`	Gateway forwards credentials to Auth Service. Auth Service verifies password via **BCrypt** and issues an **HMAC-SHA256 signed JWT token** embedding user ID and role claims.
2. Bay Scheduling & Collision Test	`API Gateway (8080)` → `Booking Service (8082)`	Gateway inspects JWT token, validates role, and routes to Booking Service. Booking Service checks bay calendar. If occupied, throws an exception mapped to **HTTP 409 Conflict**.
3. Completing a Job (Multi-Service)	`API Gateway (8080)` → `Booking Service (8082)` → `Record Service (8083)` → `Billing Service (8084)`	**Orchestrated Pipeline:** Booking Service updates status to `COMPLETED`, dispatches an OpenFeign RPC call to **Record Service** to log diagnosis, and a second OpenFeign call to **Billing Service** to calculate tax and generate the invoice.
4. Settling an Invoice ("Pay Now")	`API Gateway (8080)` → `Billing Service (8084)`	Gateway routes request to Billing Service's `PUT /bills/{id}/pay` endpoint, updating invoice state from `PENDING` to `PAID`.
Continuous (Heartbeat)	`Eureka Server (8761)` ↔ All Services	Every 30 seconds, all microservices send health heartbeats to Eureka so the Gateway and OpenFeign load-balance traffic without static IPs.

6. What Each Role Can Do on the Website

Role	Persona & Credentials	Permitted Capabilities	Enforced Restrictions (RBAC)
Customer	Fleet Operations `customer` / `cust123`	• Schedule Bay appointments (select bay, vehicle, date) • Test Bay 1 Collision Simulator (HTTP 409 check) • Cancel pending bookings (releases bay) • View Vehicle History ledger • View Invoices & click **"Pay Now"** to settle bills	× Cannot complete repair jobs × Internal service mesh hidden × Diagnostic notes entry hidden
Technician	Lead Mechanic `technician` / `tech123`	• View active Service Bay Queue • **Complete Job (Core Feature):** Open modal, enter labor ($), parts ($), diagnostics, preview 10% tax, submit → triggers OpenFeign pipeline • Review vehicle repair history	× Invoices & Billing tab hidden (confidentiality) × Cannot cancel customer bookings × Admin security inspector hidden
Admin	System Administrator `admin` / `admin123`	• **Superuser (All 5 Tabs Unlocked):** • Schedule, Cancel, and Complete bookings • View organization-wide ledger & invoices • Inspect live Eureka Service Mesh topology • Live JWT Token & Claims Inspector	✓ Full Unrestricted Superuser Access

7. Where to Find the Code in Spring Boot (Role & Task Code Reference)

Open these exact files and methods in your IDE when faculty asks for the implementation:

A. Customer Tasks

- **Bay Scheduling & Collision Avoidance:**
  File: `booking-service/src/main/java/com/precisionauto/booking/service/BookingService.java`
  Method: `public Booking scheduleBooking(BookingRequest request)`
  Endpoint: `POST /bookings` in `BookingController.java`
- **Cancel Booking:**
  File: `booking-service/src/main/java/com/precisionauto/booking/controller/BookingController.java` → `PUT /bookings/{id}/cancel`
- **View Vehicle History:**
  File: `record-service/src/main/java/com/precisionauto/record/service/RecordService.java`
  Method: `public List<ServiceRecord> getRecordsByVehicleId(String vehicleId)`
- **Pay Invoices:**
  File: `billing-service/src/main/java/com/precisionauto/billing/service/BillingService.java`
  Method: `public Billing payBill(Long billId)` (sets status to `PAID`)

B. Technician Tasks

- **Complete Job & OpenFeign Orchestration:**
  File: `booking-service/src/main/java/com/precisionauto/booking/service/BookingService.java`
  Method: `public CompleteBookingResponse completeBooking(Long id, CompleteBookingRequest req)`
  Feign Interfaces: `RecordClient.java` (`@FeignClient(name = "record-service")`) and `BillingClient.java` (`@FeignClient(name = "billing-service")`)
- **Dynamic Invoicing Calculation (Labor + Parts + 10% Tax):**
  File: `billing-service/src/main/java/com/precisionauto/billing/service/BillingService.java`
  Method: `public Billing generateBill(Billing billing)`
  Logic: `subtotal = labor.add(parts); tax = subtotal.multiply(0.10); amount = subtotal.add(tax);`

C. Security & Gateway Tasks

- **JWT Token Generation & BCrypt Auth:**
  File: `auth-service/src/main/java/com/precisionauto/auth/service/AuthService.java` → `authenticate()`
  Token Utility: `auth-service/src/main/java/com/precisionauto/auth/util/JwtUtil.java`
- **Gateway Route Filter & Token Interceptor:**
  File: `api-gateway/src/main/java/com/precisionauto/gateway/filter/JwtAuthenticationFilter.java`
  Validates Bearer token, extracts claims, and mutates request with `X-User-Id` and `X-User-Role` headers.

8. Collision Occurrence Code & Technical Implementation

1. Service Layer Implementation (Where the check happens):

```
// File: BookingService.java (Lines 44-51)
List<Booking> collisions = bookingRepository.findByBayIdAndServiceDateAndStatusNot(
        targetBay, serviceDate, "CANCELLED"
);

if (!collisions.isEmpty()) {
    throw new IllegalStateException("Collision Detected: " + targetBay +
            " is already booked for " + serviceDate + ". Please choose another bay or time.");
}
```

2. Controller Layer Implementation (Mapping to HTTP 409 Conflict):

```
// File: BookingController.java (Lines 27-38)
try {
    Booking booking = bookingService.scheduleBooking(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(booking);
} catch (IllegalStateException e) {
    return ResponseEntity.status(HttpStatus.CONFLICT).body(Map.of(
            "status", 409,
            "error", "Booking Collision",
            "message", e.getMessage()
    ));
}
```

What have you used for it? (4 Key Technical Answers for Faculty):

1. **Spring Data JPA Derived Query:** Used `findByBayIdAndServiceDateAndStatusNot(bayId, date, "CANCELLED")` in `BookingRepository` to query bay occupancy without writing raw SQL.
2. **REST Standard HTTP 409 Conflict:** Returned `HttpStatus.CONFLICT (409)` instead of generic errors to signify that the request conflicts with current state of server resources.
3. **Java 21 Streams & Predicates:** When a user leaves bay blank, Java Streams filter unoccupied bays from `ALL_BAYS = List.of("BAY-1", "BAY-2", "BAY-3")`.
4. **ACID Transaction Isolation:** Annotated with `@Transactional` to prevent race conditions during concurrent bookings.

9. Cross-Device Access (Mobile / Other Laptop) & Review Check FAQ

Q: Can I access this website on another laptop or my mobile? Will faculty test that?
Will they check that? In standard university SOA / Microservices project evaluations, faculty primarily inspect your architecture, code implementation, and live functionality directly on your presentation laptop screen. However, faculty frequently ask: "*Can this web app be accessed by technicians on their phones or tablets in the workshop?*" or occasionally test it directly.

Can you access it? YES, 100%! We have configured the frontend and microservices for seamless multi-device access:

- **Dynamic Host Resolution (`app.js`):** The frontend detects `window.location.hostname` dynamically. When opened from a phone or another laptop, all REST requests automatically target `http://<YOUR_LAPTOP_IP>:8080/api` instead of failing on `localhost`.
- **Global CORS Enabled:** The Spring Cloud API Gateway (8080) has `globalcors` configured with `allowedOriginPatterns: "*"` and `DedupeResponseHeader`, permitting cross-origin mobile and browser requests.
- **Responsive Mobile Design:** `styles.css` includes media queries for devices down to 360px width with touch-friendly cards, buttons, and input fields.

Step-by-Step Instructions to Open on Mobile / Another Laptop:

1. **Network Setup (Important):** Connect both your laptop and your mobile/second laptop to the **same Wi-Fi network** (or turn on your Mobile Hotspot on your phone and connect your laptop to it — this is the safest approach because university Wi-Fi often blocks peer-to-peer traffic via AP isolation).
2. **Start the Project:** Run `START_PROJECT_FOR_REVIEW.bat` (or `open-website.bat`).
3. **Look at the Console:** The frontend server console displays: `Mobile/Network: http://<YOUR_IP>:3000`
4. **Open Browser on Phone/Other Laptop:** Type `http://<YOUR_IP>:3000` in Chrome/Safari. The full high-tech portal will load and authenticate live!

What to say if Sir asks:

> *"Yes Sir! Our system is fully decoupled and headless. The Spring Cloud API Gateway exposes standard REST endpoints over HTTP/JSON with CORS enabled. The frontend is built with dynamic hostname resolution and responsive CSS. Workshop technicians on tablets, fleet drivers on smartphones, and admins on desktop workstations all connect seamlessly to the same centralized microservices mesh."*

10. Public Worldwide Access Link (Cloudflare Edge Tunnel)

Q: How do I generate a public link that ANYONE in the world can open on their phone or laptop?
1-Click Solution: Double-click `START_PUBLIC_LINK.bat` in the project folder.

What it does:

1. Establishes an encrypted, zero-trust edge tunnel via **Cloudflare** directly to your local web server.
2. Generates a globally accessible HTTPS link (e.g., `https://<subdomain>.trycloudflare.com`).
3. Automatically copies the link to your Windows clipboard so you can immediately paste it to Sir, WhatsApp, or email.
4. Automatically opens the link in your browser.

How the Architecture Works (Single-Origin Proxy):

- `frontend/server.js` acts as a high-performance **API Reverse Proxy**: it routes all `/api/*` requests directly to the Spring Cloud API Gateway (port 8080).
- Because both frontend assets and backend API requests flow through the same public HTTPS origin, there are **zero CORS errors**, **zero Mixed-Content security warnings**, and **zero router port forwarding required**.
- Anyone with the link (faculty in another city, on 4G/5G mobile, or on university guest Wi-Fi) can open it and test all features (login, booking, billing, diagnostics) live.

What to say to faculty if asked:

> *"Sir, our platform is designed for public cloud readiness. We have configured our Node gateway proxy to multiplex static assets and REST routes under a single origin. For this review, we route public ingress traffic through an encrypted Cloudflare Zero-Trust edge tunnel with valid SSL, allowing authorized clients worldwide to consume our microservices securely without exposing unencrypted internal service ports."*

11. Multi-Factor Authentication (Email + Password + OTP) Security Architecture

Q: How does the Email + Password + Mail OTP authentication work?

Review 2 Security Enhancement: To adhere to institutional Zero-Trust security guidelines, identity verification now uses email-based principal identification combined with mandatory email OTP verification before a signed JWT session is issued.

Two-Step Verification Pipeline:

1. **Factor 1 (Email & Password Credentials):** User submits their enterprise email (e.g., `customer@precisionauto.com`, `technician@precisionauto.com`, `admin@precisionauto.com`) and BCrypt-hashed password + Canvas CAPTCHA challenge.
2. **Factor 2 (Email OTP Token):** Upon credential validation, `auth-service` dispatches a cryptographically random 6-digit one-time authorization code to the registered email address with a 120-second TTL.
3. **Verification & Session Issuance:** User inputs the 6-digit code received in their email (or clicks the Auto-Fill from Mail button). The API Gateway issues the signed HMAC-SHA256 JWT bearer token granting dashboard access.

Backend Implementation:

- `UserRepository.java`: `findByEmail()` and `findByUsername()` derived queries.
- `AuthService.java`: `generateOtp(username)` and `verifyOtp(username, enteredOtp)` with concurrent expiration tracking.
- `AuthController.java`: `POST /auth/otp/send` and `POST /auth/otp/verify` REST endpoints.
- `server.js (Node / SMTP Engine)`: Integrated with Google SMTP (`smtp.gmail.com:587` TLS) to dispatch real, live verification emails directly to the user's inbox in real time.

What to say to Sir:

> *"Sir, for Review 2 we upgraded our identity layer to use enterprise email credentials paired with Two-Factor Authentication (2FA). When a user submits their email and password, the API Gateway authenticates the principal and triggers an automated OTP dispatch to the user's email inbox. Only when the 6-digit email code is verified does the service mesh issue the signed JWT session, providing complete Zero-Trust protection."*